

# UNION 操作在 SQLite 和 MySQL 中的实现原理研究

计算机 2303 尤颢辰      计算机 2303 杨宇

## 目录

|                                            |           |
|--------------------------------------------|-----------|
| <b>1 实验摘要</b>                              | <b>2</b>  |
| <b>2 实验概述</b>                              | <b>2</b>  |
| 2.1 实验目的 . . . . .                         | 2         |
| 2.2 实验环境 . . . . .                         | 2         |
| 2.3 实验使用的测试 SQL 以及预期结果 . . . . .           | 2         |
| 2.4 实验原理 . . . . .                         | 3         |
| <b>3 SQLite 源码分析</b>                       | <b>3</b>  |
| 3.1 一些关键的数据结构/函数说明 . . . . .               | 3         |
| 3.2 复合查询的表示 . . . . .                      | 4         |
| 3.3 multiSelect() 函数中的分流逻辑 . . . . .       | 5         |
| 3.4 multiSelectByMerge() 的排序归并逻辑 . . . . . | 5         |
| 3.5 去重字节码的实现位置 . . . . .                   | 6         |
| <b>4 SQLite 实验验证</b>                       | <b>6</b>  |
| 4.1 验证查询计划 . . . . .                       | 6         |
| 4.2 VDBE 字节码验证 . . . . .                   | 7         |
| <b>5 MySQL 源码分析</b>                        | <b>8</b>  |
| 5.1 一些关键的数据结构/函数说明 . . . . .               | 8         |
| 5.2 语法解析与查询树 . . . . .                     | 9         |
| 5.3 UNION 的临时表去重 . . . . .                 | 9         |
| 5.4 关于普通唯一键去重和哈希隐藏字段去重 . . . . .           | 10        |
| 5.5 UNION ALL 的 AppendIterator . . . . .   | 10        |
| <b>6 MySQL 实验验证</b>                        | <b>11</b> |
| <b>7 不同数据量级上的行为对比</b>                      | <b>12</b> |
| 7.1 SQLite 实验对比 . . . . .                  | 12        |
| 7.2 MySQL 实验对比 . . . . .                   | 14        |
| <b>8 SQLite 与 MySQL 的对比总结</b>              | <b>15</b> |
| <b>9 实验总结</b>                              | <b>16</b> |

## 1 实验摘要

本次实验中，我们主要围绕 SQL 集合运算中的 UNION 与 UNION ALL 展开分析，重点比较二者在执行机制上的差异。实验选取 SQLite 3.52.0 与 MySQL 8.0.45 作为分析对象，从语法解析、查询表示、执行计划、临时结构、去重算法和实验验证等角度观察这两种集合运算的实现。

实验分析表明，UNION 的语义是合并两个查询结果并去重，而 UNION ALL 只合并结果，不执行去重。SQLite 中，普通 UNION 会通过排序 + 归并来产生有序输出，并通过保存上一条输出记录，通过比较当前行与上一行来跳过重复记录。MySQL 则会创建内部临时表，通过结果列上的唯一键或添加隐藏 Hash 字段在插入时过滤重复记录。临时表在内存时默认使用 HASH 索引，落盘到 InnoDB 后则会退化为 B-Tree。

因此，UNION 的去重实现不能简单回答为“只用排序”或“只用哈希”。在本实验源码版本中，SQLite 的主线是排序归并去重，MySQL 的主线是临时表唯一键或哈希去重。实验还进一步设计了不同数据量级和行宽下的对比环节，用来观察执行时间、输出规模、临时结构和磁盘临时表是否随规模变化。

## 2 实验概述

### 2.1 实验目的

1. 理解 SQL 中 UNION 与 UNION ALL 的语义差异;
2. 分析数据库源码中这两个操作的实现流程;
3. 说明 UNION 去重发生的位置、使用的数据结构和触发条件;
4. 比较 SQLite 与 MySQL 在同一 SQL 语义下的实现差异;
5. 通过执行计划、字节码、或状态变量验证源码分析结论;
6. 观察不同数据量级和行宽条件下，UNION 与 UNION ALL 的行为差异。

### 2.2 实验环境

| 项目        | 内容                             |
|-----------|--------------------------------|
| 操作系统      | Windows11                      |
| SQLite 版本 | 3.52.0                         |
| MySQL 版本  | 8.0.45                         |
| 使用工具      | SQLite CLI、MySQL Client、VSCode |

### 2.3 实验使用的测试 SQL 以及预期结果

实验统一使用如下数据集：

```

1 CREATE TABLE t1(x INTEGER);
2 CREATE TABLE t2(x INTEGER);
3 INSERT INTO t1 VALUES (1),(2),(3);
4 INSERT INTO t2 VALUES (2),(3),(4);

```

执行的 UNION, UNION ALL 语句为:

```

1 SELECT x FROM t1 UNION SELECT x FROM t2;
2 SELECT x FROM t1 UNION ALL SELECT x FROM t2;

```

UNION 的预期输出为 (1, 2, 3, 4), UNION ALL 的预期输出为 (1, 2, 3, 2, 3, 4)。

## 2.4 实验原理

从语义上来看, UNION 的操作可以分为两步: “合并两个输入结果 + 删除重复记录”, 而 UNION ALL 只有第一步。因此, 两者的核心差异在于是否需要维护一个 “一个元素是否出现过的集合”。针对这一点, 常见的去重方式主要有两类:

1. 排序去重: 先对两个结果表排序, 使重复记录相邻, 再跳过相邻重复项实现去重。这个做法的优点是适合于 ORDER BY, 排序等操作结合执行, 缺点在于排序的过程时间复杂度  $O(N\log N)$  略高。

2. 哈希去重: 用哈希表或哈希索引记录已出现的行, 插入时发现重复则跳过。其优点为平均查找的效率高, 适合内存中的临时表。缺点则是在溢出、冲突时要进行处理, 且其对内存较为依赖。

本实验中的两个数据库体现了不同主线: 对于 SQLite, 主方案为方案 1; 对于 MySQL, 主方案为方案 2。

## 3 SQLite 源码分析

### 3.1 一些关键的数据结构/函数说明

为了避免直接进入源码时出现大量陌生名称, 本节先说明 SQLite 部分会反复使用的几个关键概念。

表 1: 复合查询相关结构和函数

| 名称            | 类型           | 在本实验中的作用                                             |
|---------------|--------------|------------------------------------------------------|
| Select        | 数据结构         | 表示一个 SELECT 查询; 复合查询中每个子查询都对应一个 Select 节点。           |
| pPrior        | Select 指针字段  | 连接多个 SELECT 节点; 在 A UNION B 中, 后一个查询可通过它找到前一个查询。     |
| op            | Select 操作符字段 | 记录集合操作类型, 如 UNION、UNION ALL、INTERSECT 或 EXCEPT。      |
| pOrderBy      | 排序条件字段       | 表示是否有全局 ORDER BY; 普通 UNION 即使未显式排序, 也可能构造排序条件进入归并去重。 |
| multiSelect() | 函数           | 复合查询处理入口; 判断查询应走排序归并、直接输出或其它集合操作路径。                  |

表 1 复合查询相关结构和函数（续）

| 名称                         | 类型         | 在本实验中的作用                                       |
|----------------------------|------------|------------------------------------------------|
| multiSelectByMerge()       | 函数         | 排序归并算法核心；普通 UNION 的去重主线会进入此函数。                 |
| SRT_Coroutine              | 结果输出方式     | 将左右子查询编译为协程，分别产生有序结果流，再交给归并逻辑处理。               |
| generateOutputSubroutine() | 函数         | 生成最终输出记录的字节码；需要去重时，会额外生成当前行与上一行的比较逻辑。          |
| regPrev                    | 虚拟机寄存器编号   | 保存上一条已输出记录；普通 UNION 用它判断重复，UNION ALL 不使用该去重逻辑。 |
| OP_Compare / OP_Jump       | VDBE 字节码指令 | 用于比较记录并按结果跳转，是 regPrev 去重逻辑的底层字节码表现。           |

之后，本章会先分析 SQLite 如何用 Select、pPrior 和 op 表示复合查询，再看 multiSelect() 如何根据 op 选择执行路径，最后进入 multiSelectByMerge() 和 generateOutputSubroutine() 解释普通 UNION 为什么能去重、UNION ALL 为什么不去重。

### 3.2 复合查询的表示

SQLite 将 A UNION B 这类复合查询组织为 Select 数据类型的链表。后一个 Select 通过 pPrior 指针指向前一个 Select，并通过 op 字段记录集合操作类型，例如 TK\_UNION (指 UNION 操作) 或 TK\_ALL (指 UNION ALL 操作)。

```
const char *sqlite3SelectOpName(int id){
    char *z;
    switch( id ){
        case TK_ALL:      z = "UNION ALL";    break;
        case TK_INTERSECT: z = "INTERSECT";  break;
        case TK_EXCEPT: z = "EXCEPT";    break;
        default:          z = "UNION";       break;
    }
    return z;
}
```

图 1: 不同集合操作的字段名 (select.c 1626-1631)

处理复合查询的入口为 multiSelect() 函数，若 pPrior 指针有指向，这个查询就是一个复合查询。

```
#ifndef SQLITE_OMIT_COMPOUND_SELECT
/* Handle compound SELECT statements using the separate multiSelect()
** procedure.  tag-select-0300
*/
if( p->pPrior ){
    rc = multiSelect(pParse, p, pDest);
}
```

图 2: 普通 SELECT 转入 multiselect (select.c 7881-7885)

所以，SQLite 不是为 UNION 单独写一个执行器，而是先把多个 SELECT 操作串成一个查询链，再由 multiSelect() 根据操作符对不同集合操作进行分流处理。

### 3.3 multiSelect() 函数中的分流逻辑

转入复合查询之后，multiSelect() 会根据复合查询的排序要求和集合操作类型选择执行路径。只要复合查询带有全局 ORDER BY，也就是本来就需要做排序的要求，SQLite 就需要按照统一顺序合并左右子查询结果，因此会调用 multiSelectByMerge()，即使用排序归并算法进行处理。如果查询不是 UNION ALL，也就是普通 UNION、INTERSECT 或 EXCEPT，SQLite 即使没有看到用户写出的 ORDER BY，也会在内部构造一个排序条件 (ORDER BY 1)，使结果能够进入归并处理流程。只有在查询是 UNION ALL 且没有全局 ORDER BY 时，SQLite 才可以省去排序归并过程，直接先执行左查询，再执行右查询，并把两边结果连续输出。

下面是分流逻辑的源码，分别对应上面的三种逻辑：

```
if( p->pOrderBy ){
    /* If the compound has an ORDER BY clause, then always use the merge
    ** algorithm. */
    return multiSelectByMerge(pParse, p, pDest);
}else if( p->op!=TK_ALL ){
    /* If the compound is EXCEPT, INTERSECT, or UNION (anything other than
    ** UNION ALL) then also always use the merge algorithm. However, the
    ** multiSelectByMerge() routine requires that the compound have an
    ** ORDER BY clause, and it doesn't right now. So invent one first. */
    Expr *pOne = sqlite3ExprInt32(db, 1);
    p->pOrderBy = sqlite3ExprListAppend(pParse, 0, pOne);
    if( pParse->nErr ) goto multi_select_end;
    assert( p->pOrderBy!=0 );
    p->pOrderBy->a[0].u.x.iOrderByCol = 1;
    return multiSelectByMerge(pParse, p, pDest);
}else{
    /* For a UNION ALL compound without ORDER BY, simply run the left
    ** query, then run the right query */

```

图 3: 分流逻辑 (select.c 2980-2997)

由此可知，SQLite 3.52.0 中普通 UNION 的处理逻辑不是建立去重表，而是主动去做排序归并。事实上，这个处理逻辑是从 2026 年初开始运用于 SQLite 的。

### 3.4 multiSelectByMerge() 的排序归并逻辑

multiSelectByMerge() 会将左右两个子查询分别编译为协程，并让两个子查询按照同一排序规则产生有序输出。归并时，SQLite 持续比较左右两边的当前行：如果左边当前行更小，就输出左边记录；如果右边当前行更小，就输出右边记录；如果两边当前行相等则只输出其中一份，从而避免重复结果进入最终输出。

由于源码较长，在此仅列出代码给出的描述排序归并的注释：

|       | UNION ALL   | UNION       | EXCEPT      | INTERSECT   |
|-------|-------------|-------------|-------------|-------------|
| AltB: | outA, nextA | outA, nextA | outA, nextA | nextA       |
| AeqB: | outA, nextA | nextA       | nextA       | outA, nextA |
| AgB:  | outB, nextB | outB, nextB | nextB       | nextB       |
| EofA: | outB, nextB | outB, nextB | halt        | halt        |
| EofB: | outA, nextA | outA, nextA | outA, nextA | halt        |

图 4: 排序归并逻辑

源码注释中的 AltB 表示 A 当前行小于 B 当前行，AeqB 表示二者相等，AgtB 表示 A 当前行大于 B 当前行；EofA 和 EofB 分别表示 A 或 B 已经读完。表中的 outA / outB 表示输出 A 或 B 的当前行，nextA / nextB 表示读取该输入流的下一行，halt 表示当前集合运算已经可以结束。

所以对于 UNION ALL，只要某一侧还有记录，就直接输出，因此相等时也会输出 A 并继续推进，重复行不会被删除。对于普通 UNION，当 A 和 B 当前行相等时，SQLite 不会同时输出两份，而是只推进其中一侧，使重复值在归并过程中只进入结果一次。

### 3.5 去重字节码的实现位置

真正生成跳过重复行的字节码的位置在 generateOutputSubroutine() 函数中。当 regPrev > 0 时，SQLite 会生成比较当前行和上一条输出行的字节码；如果二者相等，就跳过当前行，否则复制当前行到 regPrev 并输出。下面是具体的代码实现：

```
if( regPrev ){
    int addr1, addr2;
    addr1 = sqlite3VdbeAddOp1(v, OP_IfNot, regPrev); VdbeCoverage(v);
    addr2 = sqlite3VdbeAddOp4(v, OP_Compare, pIn->iSdst, regPrev+1, pIn->nSdst,
        (char*)sqlite3KeyInfoRef(pKeyInfo), P4_KEYINFO);
    sqlite3VdbeAddOp3(v, OP_Jump, addr2+2, iContinue, addr2+2); VdbeCoverage(v);
    sqlite3VdbeJumpHere(v, addr1);
    sqlite3VdbeAddOp3(v, OP_Copy, pIn->iSdst, regPrev+1, pIn->nSdst-1);
    sqlite3VdbeAddOp2(v, OP_Integer, 1, regPrev);
}
```

图 5: 去重逻辑 (select.c 3118-3127)

因此，regPrev 可以看作 SQLite 判断是否生成去重逻辑的开关。普通 UNION 会在 multiSelectByMerge() 中分配并使用 regPrev，并在 generateOutputSubroutine() 中生成“当前行与上一条输出行比较，重复则跳过”的字节码；UNION ALL 则把 regPrev 设为 0，不生成这段逻辑。这也是 SQLite 中这两者最直接的实现差异。

## 4 SQLite 实验验证

### 4.1 验证查询计划

执行命令：

```
1 .eqp on
2 SELECT x FROM t1 UNION SELECT x FROM t2;
3 SELECT x FROM t1 UNION ALL SELECT x FROM t2;
```

对于 UNION 查询，结果如下：

```
QUERY PLAN
|--MERGE (UNION)
|  |--LEFT
|  |  |--SCAN t1
|  |  '--USE TEMP B-TREE FOR ORDER BY
|--RIGHT
|  |--SCAN t2
|  '--USE TEMP B-TREE FOR ORDER BY
```

图 6: UNION Result



对 UNION ALL，字节码中没有出现排序器相关指令，也没有出现用于比较上一条输出记录的去重指令组合。这说明 UNION ALL 不进行重复检查，而是直接保留左右子查询的全部结果。

```

sqlite> sqlite> EXPLAIN SELECT x FROM t1 UNION ALL SELECT x FROM t2;
addr  opcode      p1  p2  p3  p4  p5  comment
-----
0     Init       0   12  0   0   0   Start at 12
1     OpenRead   1   2   0   1   0   root=2 iDb=0; t1
2     Rewind     1   6   0   0   0
3     Column     1   0   1   0   0   r[1]= cursor 1 column 0
4     ResultRow  1   1   0   0   0   output=r[1]
5     Next       1   3   0   0   1
6     OpenRead   0   3   0   1   0   root=3 iDb=0; t2
7     Rewind     0   11  0   0   0
8     Column     0   0   1   0   0   r[1]= cursor 0 column 0
9     ResultRow  1   1   0   0   0   output=r[1]
10    Next       0   8   0   0   1
11    Halt       0   0   0   0   0
12    Transaction 0   0   2   0   1   usesStmtJournal=0
13    Goto       0   1   0   0   0

```

图 9: VDBE Byte Code(UNION ALL)

## 5 MySQL 源码分析

### 5.1 一些关键的数据结构/函数说明

接下来我们对比分析一下 MySQL 的集合运算实现逻辑。我们还是先介绍一下之后会用到的一些数据结构与函数名：

表 2: MySQL 复合查询相关结构和函数

| 名称                        | 类型       | 在本实验中的作用                                                   |
|---------------------------|----------|------------------------------------------------------------|
| PT_union                  | 解析树节点    | 表示解析阶段识别出的 UNION 结构；它不是最终执行器，而是从 SQL 文本到查询表达式的中间形态。        |
| PT_union::contextualize() | 函数       | 对 UNION 解析树节点做语义处理，将语法层面的集合操作转换为后续优化和执行可用的查询结构。            |
| Query_term_union          | 数据结构 / 类 | MySQL 表示 UNION 查询项的核心结构；查询树中识别 UNION 主要就是指这类结构。            |
| ::create_result_table()   | 函数       | 为 UNION 结果创建内部临时表；普通 UNION 需要去重，因此会把 distinct 信息传给临时表创建逻辑。 |
| create_tmp_table()        | 函数       | 真正构造内部临时表结构；决定使用普通唯一键，或在必要时添加隐藏 <hash_field> 辅助去重。         |
| <auto_distinct_key>       | 内部唯一键名称  | MySQL 在临时表结果列上建立的唯一键，用于阻止重复记录插入。                           |
| <hash_field>              | 隐藏字段     | 结果列太长、列太多或集合操作较复杂时，MySQL 可能添加隐藏哈希字段，用整行哈希值辅助判重。            |

表 2 MySQL 复合查询相关结构和函数（续）

| 名称                        | 类型       | 在本实验中的作用                                          |
|---------------------------|----------|---------------------------------------------------|
| HA_KEY_ALG_HASH           | 索引算法标识   | 表示内存引擎默认使用 HASH 索引；解释了 MySQL 内存临时表去重常被概括为哈希去重的原因。 |
| ha_write_row()            | 存储引擎写入接口 | 执行阶段向临时表写入一行记录；若该行违反唯一性约束，重复记录会被忽略。               |
| check_unique_constraint() | 函数       | 在隐藏哈希字段场景下检查记录是否真正重复，避免只比较哈希值导致哈希冲突误判。            |
| AppendIterator            | 执行器迭代器   | 用于顺序读取多个子查询结果；简单 UNION ALL 不需要去重，常通过它串接多个结果流输出。   |

之后，本章会先分析 MySQL 如何在语法和查询结构中表示 UNION，再看普通 UNION 如何通过 `create_result_table()` 和 `create_tmp_table()` 创建可去重的内部临时表，最后说明重复行如何在写入临时表时被过滤，以及 UNION ALL 为什么可以使用更直接的 `AppendIterator` 路径。

## 5.2 语法解析与查询树

MySQL 将 UNION、EXCEPT、INTERSECT 作为集合操作处理。语法解析后会构造对应的查询项对象，UNION 对应的查询项对象是 `Query_term_union`。

```

/// Node type for n-ary UNION
class Query_term_union : public Query_term_set_op {
public:
    /**
     * Constructor.
     * @param mem_root      the mem_root to use for allocation
     */
    Query_term_union(MEM_ROOT *mem_root) : Query_term_set_op(mem_root) {}
    Query_term_type term_type() const override { return QT_UNION; }
    const char *operator_string() const override { return "union"; }
    void debugPrint(int level, std::ostringstream &buf) const override;
};

```

图 10: 集合操作类定义 (sql/query\_term.h 495-503)

MySQL 在解析 SQL 时，会把 UNION 这类集合运算转换为内部查询树结构。这个结构不仅保存左右两个子查询，还保存集合操作的类型，例如普通 UNION 或 UNION ALL。后续优化和执行阶段会根据这些信息决定执行方式。

## 5.3 UNION 的临时表去重

MySQL 的 UNION 操作需要将结果写入内部临时表，并通过临时表的唯一约束过滤重复记录。

执行 UNION 时，MySQL 会先为集合运算准备一张内部临时结果表，然后把各个子查询产生的记录写入这张临时表。在临时表上会做唯一性检查：如果某一行此前没有出现过，就可以成功写入；如果同一行已经存在，新的插入会被识别为重复并被忽略。最终，MySQL 再从这张临时表中读取已经去重的结果。

```

bool Query_result_union::create_result_table(
    THD *thd_arg, const mem_root_deque<Item *> &column_types,
    bool is_union_distinct, ulonglong options, const char *table_alias,
    bool bit_fields_as_long, bool create_table, Query_term_set_op *op) {
    mem_root_deque<Item *> visible_fields(thd_arg->mem_root);
    for (Item *item : Visiblefields(column_types)) {
        visible_fields.push_back(item);
    }
}

```

图 11: 创建结果临时表 (sql/sql\_union.cc 168-175)

```

if (! (table = create_tmp_table(thd_arg, &tmp_table_param, visible_fields,
                                nullptr, is_union_distinct, true, options,
                                HA_POS_ERROR, table_alias)))
    return true;
if (create_table) {
    table->file->ha_extra(HA_EXTRA_IGNORE_DUP_KEY);
    if (table->hash_field) table->file->ha_index_init(0, false);
}
return false;
}

```

图 12: 将是否去重的信息传给临时表

## 5.4 关于普通唯一键去重和哈希隐藏字段去重

与 SQLite 不同, MySQL 的去重不依赖“输出时比较上一行”, 而是在写入临时表时通过唯一约束完成。这个唯一约束可能直接建立在结果列上, 也可能借助隐藏 `<hash_field>` 先快速定位可能重复的记录, 再通过完整行比较避免哈希冲突造成误判。

下面是这两种方式的对比:

表 3: MySQL 临时表去重实现路径

| 路径                              | 使用条件                                               | 实现方式                                           |
|---------------------------------|----------------------------------------------------|------------------------------------------------|
| 普通唯一键                           | 结果列可以直接建唯一键                                        | 在结果列上创建 <code>&lt;auto_distinct_key&gt;</code> |
| <code>&lt;hash_field&gt;</code> | key 太长、列太多、特殊集合操作或混合 <code>DISTINCT/ALL</code> 等情况 | 增加隐藏哈希字段, 用整行哈希辅助判重                            |

下面展示的是代码中如何判断是否需要 hash 隐藏字段, 如果结果列组成的唯一键过长、列数过多, 或者调用方强制要求使用哈希字段, 源码会建立 hash 隐藏字段。

```

if (distinct) {
    if (distinct_key_length > max_key_length ||
        (fieldnr - param->hidden_field_count) > max_key_parts ||
        param->force_hash_field_for_unique) {
        unique_constraint_via_hash_field = true;
    }
}
}

```

图 13: 判断是否需要 hash 隐藏字段 (sql/sql\_tmp\_table.cc 1368-1373)

之后, MySQL 在执行阶段会将子查询结果写入临时表, 对于 UNION 操作, 写入时如果发现重复键, 就忽略重复行, 这样就实现了实现去重的功能。值得一提的是, 在存在隐藏 hash 字段时, MySQL 不能只比较哈希值, 因为不同记录可能出现哈希冲突。因此源码中还需要执行完整记录比较, 保证不会误判。

## 5.5 UNION ALL 的 AppendIterator

UNION ALL 不要求去重, 因此 MySQL 可以在简单场景下使用 AppendIterator 把多个子查询的输出首尾相接。需要注意的是, UNION ALL 不去重并不等于永远不使用临时表。如果查询带有全局排序、复杂混合集合操作或其它执行要求, MySQL 仍可能使用临时表, 但这不是为了 ALL 的语义。

```

/**
 * Takes in two or more iterators and output rows from them sequentially
 * (first all rows from the first one, the all from the second one, etc.).
 * Used for implementing UNION ALL, typically together with StreamingIterator.
 */
class AppendIterator final : public RowIterator {
public:
    AppendIterator(
        THD *thd,
        std::vector<unique_ptr<destroy_only<RowIterator>>> &&sub_iterators);

```

图 14: AppendIterator 用于多个子查询的输出首尾相接

## 6 MySQL 实验验证

下面我们验证一下执行计划是不是和我们分析的相同。

执行命令：

```

1 EXPLAIN FORMAT=TREE
2 SELECT x FROM t1 UNION SELECT x FROM t2;
3
4 EXPLAIN FORMAT=TREE
5 SELECT x FROM t1 UNION ALL SELECT x FROM t2;

```

对于 UNION 查询，结果如下：

```

+-----+
| EXPLAIN
+-----+
| -> Table scan on <union temporary> (cost=2.13..4.28 rows=6)
|   -> Union materialize with deduplication (cost=1.7..1.7 rows=6)
|     -> Table scan on t1 (cost=0.55 rows=3)
|     -> Table scan on t2 (cost=0.55 rows=3)
|
+-----+
1 row in set (0.00 sec)

```

图 15: EXPLAIN UNION

我们可以看到，执行计划中出现 materialize / deduplication 相关信息，说明 MySQL 会将多个子查询结果写入内部临时结果结构，并在该结构中完成去重。

对于 UNION ALL 查询，结果如下：

```

+-----+
| EXPLAIN
+-----+
| -> Append (cost=1.1 rows=6)
|   -> Stream results (cost=0.55 rows=3)
|     -> Table scan on t1 (cost=0.55 rows=3)
|   -> Stream results (cost=0.55 rows=3)
|     -> Table scan on t2 (cost=0.55 rows=3)
|
+-----+

```

图 16: EXPLAIN UNION ALL

我们看到，计划是 Append 形式的，即顺序读取左右子查询结果并输出，不需要唯一性检查。

## 7 不同数据量级上的行为对比

前面的源码分析回答了“数据库如何实现”的问题。本节进一步回答“当数据量变大时，这套实现会表现出什么行为差异”的问题。因此，本节把数据规模和行宽作为实验变量，观察执行时间、输出行数、执行计划和临时结构的变化。

本实验从两个维度改变数据：

表 4: 实验变量设计

| 变量       | 取值                                | 观察目的                     |
|----------|-----------------------------------|--------------------------|
| 数据规模 $N$ | 小: $10^3$ ; 中: $10^5$ ; 大: $10^6$ | 数据库处理 UNION 的额外成本会不会明显变大 |
| 行宽       | 窄行: 单个整数列; 宽行: 整数列加较长字符串列         | 观察数据库内部临时结构的压力变大如何处理     |

### 7.1 SQLite 实验对比

对 SQLite 部分, 我们主要改变数据量级  $N$ , 而不展开行宽的实验。原因在于 SQLite 3.52.0 中普通 UNION 的核心执行路径较稳定, 主要表现为排序归并和输出阶段去重。行宽主要影响排序记录和比较成本, 其不会像 MySQL 那样触发明显的策略切换。

所以, 我们固定表结构, 改变数据量级, 按照下面的顺序进行实验:

打开终端, 执行:

```
1 C:\msys64\mingw64\bin\sqlite3.exe sqlite_union_scale.db
2 .timer on
3 .eqp on
```

这一步我们开启了计时器来记录耗费时长, 并观察查询计划。为了避免大量结果输出影响实验时间, 查询外层统一使用 COUNT(\*) 统计输出行数。

对于  $N = 10^3, N = 10^5, N = 10^6$ , 分别执行下面的一系列操作:

清空旧表 + 建表:

```
1 DROP TABLE IF EXISTS t1;
2 DROP TABLE IF EXISTS t2;
3 CREATE TABLE t1(x INTEGER);
4 CREATE TABLE t2(x INTEGER);
```

插入数据 (单列表结构、下面样例代码是  $N = 10^3$  的情况):

```
1 WITH RECURSIVE seq(x) AS (
2     SELECT 1
3     UNION ALL
4     SELECT x + 1 FROM seq WHERE x < 1000
5 )
6 INSERT INTO t1 SELECT x FROM seq;
7 WITH RECURSIVE seq(x) AS (
8     SELECT 501
```

```

9  UNION ALL
10 SELECT x + 1 FROM seq WHERE x < 1500
11 )
12 INSERT INTO t2 SELECT x FROM seq;

```

确认数据量：

```

1 SELECT COUNT(*) FROM t1;
2 SELECT COUNT(*) FROM t2;

```

分别执行 UNION 和 UNION ALL：

```

1 SELECT COUNT(*) FROM (
2   SELECT x FROM t1
3   UNION
4   SELECT x FROM t2
5 );
6 SELECT COUNT(*) FROM (
7   SELECT x FROM t1
8   UNION ALL
9   SELECT x FROM t2
10 );

```

对于不同的自变量，得到的结果如下表：

表 5: SQLite 不同数据量下 UNION 与 UNION ALL 的执行结果

| 数据量级 $N$ | 查询类型      | 输出行数    | 执行时间 real | 关键执行计划                                      |
|----------|-----------|---------|-----------|---------------------------------------------|
| $10^3$   | UNION     | 1500    | 0.038s    | MERGE (UNION), USE TEMP B-TREE FOR ORDER BY |
| $10^3$   | UNION ALL | 2000    | 0.015s    | SCAN t1, SCAN t2, 无去重归并结构                   |
| $10^5$   | UNION     | 150000  | 0.134s    | MERGE (UNION), USE TEMP B-TREE FOR ORDER BY |
| $10^5$   | UNION ALL | 200000  | 0.029s    | SCAN t1, SCAN t2, 无去重归并结构                   |
| $10^6$   | UNION     | 1500000 | 0.996s    | MERGE (UNION), USE TEMP B-TREE FOR ORDER BY |
| $10^6$   | UNION ALL | 2000000 | 0.094s    | SCAN t1, SCAN t2, 无去重归并结构                   |

从实验结果可以看出，在三个数据量级下，UNION 的输出行数分别为  $1.5N$ ，UNION ALL 的输出行数分别为  $2N$ ，说明普通 UNION 执行了去重，而 UNION ALL 保留了重复记录。查询计划显示，普通 UNION 始终出现 MERGE (UNION) 和 USE TEMP B-TREE FOR ORDER BY，说明 SQLite 在不同数据量级下都采用排序归并路径；而 UNION ALL 只扫描两张表，没有出现去重归并结构。随着  $N$  的增长，UNION 的执行时间从 0.038s 增加到 0.996s，增长明显大于 UNION ALL，说明数据量增大会放大普通 UNION 中排序、归并和去重带来的额外成本。

另外需要注意的是，如果数据规模再继续增长，UNION 带来的查询压力就基本是不可接受的了。SQLite 不会自动将超大规模 UNION 查询改写为分批处理、增量去重或预建索引等优化方案，用户需要自己去完成一些措施来解决。

## 7.2 MySQL 实验对比

对于 MySQL，两个变量我们都做了对比实验。因为普通 UNION 的去重依赖内部临时表和唯一性检查。数据量级影响临时表写入成本，行宽影响临时表记录大小以及是否更容易触发隐藏 `<hash_field>` 或磁盘临时表。因此，这两个变量分别对应 MySQL 去重实现中的规模成本和临时表存储压力。限于篇幅原因，具体的实验使用的指令就不列出了，我们重点看输出的结果对比。

首先，我们固定行宽为窄行，观察查询运行时间，结果如下表：

表 6: MySQL 不同数据量下 UNION 与 UNION ALL 的执行结果

| 数据量级 $N$ | 查询类型      | 输出行数    | 执行时间  | 关键执行计划                                                                |
|----------|-----------|---------|-------|-----------------------------------------------------------------------|
| $10^3$   | UNION     | 1500    | 0.00s | Table scan on <union temporary>; Union materialize with deduplication |
| $10^3$   | UNION ALL | 2000    | 0.00s | Append; Stream results                                                |
| $10^5$   | UNION     | 150000  | 0.35s | Table scan on <union temporary>; Union materialize with deduplication |
| $10^5$   | UNION ALL | 200000  | 0.21s | Append; Stream results                                                |
| $10^6$   | UNION     | 1500000 | 8.93s | Table scan on <union temporary>; Union materialize with deduplication |
| $10^6$   | UNION ALL | 2000000 | 3.96s | Append; Stream results                                                |

从执行计划看，普通 UNION 在三个数据量级下都出现 `<union temporary>` 和 `Union materialize with deduplication`，说明 MySQL 始终采用内部临时表 + 去重的执行方式。UNION ALL 则始终出现 `Append` 和 `Stream results`，说明它不需要去重，可以直接串接两个子查询结果。

从执行时间看，随着数据量从增加，两类查询耗时都增加，但 UNION 的增长更明显。 $N = 10^6$  时，UNION 用时 8.93s，而 UNION ALL 用时 3.96s，说明内部临时表写入和去重检查会在大数据量下显著放大普通 UNION 的成本。

综合来看，数据量级不会改变 MySQL 对 UNION / UNION ALL 的基本执行路线，但会放大两者的性能差异，这一点和 SQLite 是一样的。

值得注意的是，当内存临时表不足以容纳结果时，MySQL 可以将临时表转换为磁盘 InnoDB 临时表继续执行。也就是说，MySQL 的内部临时表机制比 SQLite 更完整，但超大规模 UNION 仍需要用户主动优化。

其次，我们固定  $N = 10^5$ ，观察宽记录加重 MySQL 内部临时表压力会发生什么：

```
mysql> SHOW SESSION STATUS LIKE 'Created_tmp_tables';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Created_tmp_tables | 20 |
+-----+-----+
1 row in set (0.02 sec)

mysql> SHOW SESSION STATUS LIKE 'Created_tmp_disk_tables';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Created_tmp_disk_tables | 2 |
+-----+-----+
1 row in set (0.00 sec)
```

图 17: 窄表

```
mysql> SHOW SESSION STATUS LIKE 'Created_tmp_tables';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Created_tmp_tables | 22 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SHOW SESSION STATUS LIKE 'Created_tmp_disk_tables';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Created_tmp_disk_tables | 2 |
+-----+-----+
1 row in set (0.00 sec)
```

图 18: 窄表

```
mysql> SHOW SESSION STATUS LIKE 'Created_tmp_tables';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Created_tmp_tables | 24 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SHOW SESSION STATUS LIKE 'Created_tmp_disk_tables';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Created_tmp_disk_tables | 2 |
+-----+-----+
1 row in set (0.00 sec)
```

图 19: 窄表

```
mysql> SHOW SESSION STATUS LIKE 'Created_tmp_tables';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Created_tmp_tables | 26 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SHOW SESSION STATUS LIKE 'Created_tmp_disk_tables';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Created_tmp_disk_tables | 3 |
+-----+-----+
1 row in set (0.00 sec)
```

图 20: 窄表

我们可以看到，从窄表变为宽表之后，查询的执行时间从 0.34s 增加到 3.61s，说明行宽显著增加了内部临时表写入、比较和去重成本。同时，窄行实验中 Created\_tmp\_disk\_tables 没有增加，而宽行实验中该值增加了 1，说明宽记录更容易使内部临时表落盘。这说明，MySQL 的普通 UNION 依赖内部临时表和唯一性检查完成去重，行宽越大，临时表存储压力越明显。

## 8 SQLite 与 MySQL 的对比总结

下面，我们对 SQLite 和 MySQL 在实现这两种集合操作的实现上进行一个总结：

表 7: SQLite 与 MySQL 中 UNION 实现方式对比

| 对比维度           | SQLite 3.52.0 | MySQL 8.0.45             |
|----------------|---------------|--------------------------|
| 普通 UNION 实现路径  | 排序归并          | 内部临时表                    |
| 去重方式           | 通过比较，跳过重复输出   | 通过唯一键或哈希辅助结构，在写入临时表时过滤重复 |
| UNION ALL 实现路径 | 直接执行左右查询并输出   | 使用 Append 串接结果流          |
| 数据量增大后的表现      | 排序、归并和比较成本上升  | 临时表写入、唯一性检查和可能的落盘成本上升    |
| 大数据场景          | 不会自动分批，需主动优化  | 有更完整的临时表机制，但仍需用户主动优化     |

## 9 实验总结

这次试验，我们围绕 UNION 和 UNION ALL 在 SQLite 与 MySQL 中的实现方式展开，对两种数据库的集合查询执行机制进行了对比分析。

从实现路径来看，SQLite 对普通 UNION 使用排序归并方式。它会借助临时 B-Tree 形成有序结果流，再通过比较相邻记录完成去重；而 UNION ALL 不需要去重，因此可以直接扫描左右子查询并输出。MySQL 则使用内部临时表策略。普通 UNION 会将结果写入临时表，并通过唯一键或隐藏哈希字段辅助去重；UNION ALL 则通常使用 Append 流式输出方式串接多个子查询结果。

从实验结果来看，UNION ALL 在两种数据库中都明显快于普通 UNION。原因在于 UNION ALL 不需要执行去重逻辑，避免了排序、归并、唯一性检查或临时表写入等额外成本。随着数据规模增大，普通 UNION 的耗时增长更加明显，说明去重是集合查询中的主要性能开销之一。

SQLite 和 MySQL 的差异也体现了两种系统设计取向的不同。SQLite 的实现结构相对轻量，执行计划中可以直接看到 MERGE (UNION) 和 USE TEMP B-TREE 等特征；MySQL 的执行机制更复杂，涉及解析树节点、查询项结构、临时表创建、存储引擎写入接口等。MySQL 在大数据场景下具有更完整的临时表机制，但也会带来一些额外成本。

综合来看，如果业务语义允许保留重复记录，应优先使用 UNION ALL，避免不必要的去重开销；只有在确实需要去除重复结果时，才使用普通 UNION。对于大数据量查询，还应结合需求进行优化。通过本实验可以更清楚地看到，SQL 表面上相似的集合操作，在数据库内部可能对应完全不同的执行路径，理解这些底层机制帮助我们更合理地编写和优化查询语句。